Alarm domowy Cerber 1.0

Creative Commons Zero (CC0)

Wstęp

Masz przed sobą dokumentację techniczną prostego alarmu domowego. Alarm może być nieuzbrojony lub uzbrojony. Można go uzbrajać, zmieniać hasło, przywracać ustawienia fabryczne.

Urządzenie korzysta z czujnika ruchu. Buzzer i dioda LED służą do sygnalizacji stanów urządzenia. Urządzenie korzysta z wbudowanej pamięci EEPROM, dzięki czemu nawet przy utracie prądu – pamięta, że było uzbrojone i jakie było ustawione hasło. Dzięki wykorzystaniu modułu RTC w logach urządzenia widzimy dokładne daty i godziny poszczególnych zdarzeń.

W projekcie został wykorzystany Arduino UNO R4 WiFi, jednak wystarczy zwykłe Arduino UNO bądź jego odpowiednik.

Założenia projektowe

- możliwość uzbrojenia alarmu
- zmiana hasła użytkownika
- wykrywanie ruchu PIR
- alarm dźwiękowy
- sygnalizacja RGB
- zapis stanu po zaniku zasilania
- obsługa RTC
- obsługa LCD
- obsługa klawiatury matrycowej

Wykorzystane komponenty

Element	Model	Funkcja	Ilość sztuk
Arduino	UNO R4 WiFi	Sterownik główny	1
PIR	HC-SR501	Detekcja ruchu	1
RTC	DS1307 / DS3231	Data i godzina	1
LCD	1602 I2C	Interfejs użytkownika	1
Dioda RGB LED	wspólna katoda	Sygnalizacja stanu	1
Keypad	4x4	Wprowadzanie danych	1
EEPROM	wbudowany w płytkę Arduino	Przechowywanie danych pomimo utraty zasilania	1
Rezystory	330 Om	Ochrona przed spaleniem komponentów	4

Buzzer	buzzer z generatorem 5V 12mm - THT	Dźwięki alarmu	1
Płytka stykowa	Duża	Połączenie elementów w układ	1
Kable połączeniowe	Różne długości, wejścia męsko – męskie, męsko - żeńskie	Połączenia elektryczne pomiędzy elementami układu	1

Schemat połączeń

Pogrupowane po urządzeniach

Pin Arduino	**Urządzenie**	**Pin urządzenia**
A0	Keypad	R4
A1	Keypad	R3
A2	Keypad	R2
A3	Keypad	R1
D2	Keypad	C1
D3	Keypad	C2
D4	Keypad	C3
D5	Keypad	C4
A4 / SDA	LCD, RTC	SDA
A5 / SCL	LCD, RTC	SCL
5V	Zasilanie	
GND	Uziemienie	
D13	Buzzer	+
D12	PIR	Out
D11 (PWM)	LED	Red
D10 (PWM)	LED	Green
D9 (PWM)	LED	Blue

Obsługa EEPROM

Adres	**Dane**
0-9	Hasło
100	Stan alarmu

Wymagane biblioteki

- RTClib
- LiquidCrystal I2C

Stany alarmu

- WYLACZONY,
- UZBROJONY,
- WYKRYTO_RUCH,
- ALARM,
- ZMIANA_HASLA

Znaczenie klawiszy

Klawisz	**Funkcja**
0-9	Cyfry
*	Informacja o funkcjach przycisków A-D
#	Zatwierdzenie wpisanych znaków lub powrót do głównego menu
A	Uzbrajanie (dostępny tylko w stanie WYLACZONY)
B	Zmiana hasła (dostępny tylko w stanie WYLACZONY)
C	Kasowanie wprowadzonych znaków
D	Przywrócenie ustawień fabrycznych (dostępny tylko w stanie WYLACZONY)

Schemat działania alarmu

Domyślne hasło to 1234.

Domyślnie alarm jest nieuzbrojony - stan WYLACZONY.

W tym stanie użytkownik może: uzbroić alarm, zmienić hasło, przywrócić do ustawień fabrycznych, sprawdzić informację o znaczeniu klawiszy. Dioda świeci ciągłym białym światłem symbolizując gotowość do pracy.

Wciskając przycisk A – alarm przechodzi w stan UZBROJONY, następuje aktywacja czujnika ruchu PIR oraz pojawia się ekran z możliwością wpisania hasła. Dioda mruga na żółto.

Jeśli zostanie wykryty ruch – alarm przechodzi w stan WYKRYTO_RUCH, a użytkownik ma 10 sekund na wpisanie poprawnego hasła.

Wpisanie nieprawidłowego hasła lub przekroczenie czasu powoduje stan ALARM, w którym mruga czerwona dioda i jest odtwarzany dźwięk z buzzera.

Jeśli użytkownik wpisze poprawne hasło i zatwierdzi krzyżykiem – alarm przechodzi w stan WYLACZONY.

Wciskając przycisk B – alarm przechodzi w stan ZMIANA_HASLA, podczas którego użytkownik może zmienić hasło. Nowe hasło musi składać się wyłącznie z cyfr i mieć minimalnie 4 znaki. Kratka zatwierdza hasło, przycisk C - kasuje dotychczas wpisane znaki.

W przypadku zbyt krótkiego hasła – zostaje wyświetlony komunikat o błędzie.

Wciskając przycisk D – ustawienia alarmu są przywracane do ustawień fabrycznych, co oznacza również reset hasła do ciągu 1234.

Wciskając przycisk * – na ekranie pojawia się legenda objaśniająca działanie klawiszy A, B, C, D.

Scenariusze testowe

Scenariusz A

1. Stan alarmu: WYLACZONY.
2. Wciśnij A – alarm przechodzi w stan UZBROJONY.
3. Wpisz nieprawidłowe hasło, zatwierdź #.
4. Uruchamiany jest sygnał dźwiękowy i dioda miga na czerwono.
5. Wpisz poprawne hasło.
6. Alarm wraca do stanu WYLACZONY.

Scenariusz B

1. Stan alarmu: WYLACZONY.
2. Wciśnij A – alarm przechodzi w stan UZBROJONY.
3. Wpisz prawidłowe hasło.
4. Alarm wraca do stanu WYLACZONY.

Scenariusz C

1. Stan alarmu: WYLACZONY.
2. Wciśnij B, aby zmienić hasło.
3. Wprowadź zbyt krótkie hasło, zatwierdź #.
4. Na wyświetlaczu pokaże się ostrzeżenie.
5. Wprowadź kilka znaków i wciśnij C, aby je skasować.
6. Wprowadź poprawne hasło, zatwierdź #.
7. Na wyświetlaczu pokaże się informacja o zmienionym haśle.
8. Wciśnij A, aby uzbroić alarm.
9. Odłącz zasilanie.
10. Podłącz ponownie zasilanie.
11. Alarm pamięta ostatni stan i uruchomi się w stanie UZBROJONY.
12. Spróbuj odblokować fabrycznym hasłem 1234 i zatwierdź #.
13. Alarm uruchomi się.

14. Wprowadź właściwe hasło.
15. Alarm wraca do stanu WYLACZONY.

Scenariusz D

1. Stan alarmu: WYLACZONY.
2. Wciśnij D – urządzenie jest resetowane do ustawień fabrycznych.
3. Wciśnij #, aby wrócić do ekranu głównego.

Scenariusz E

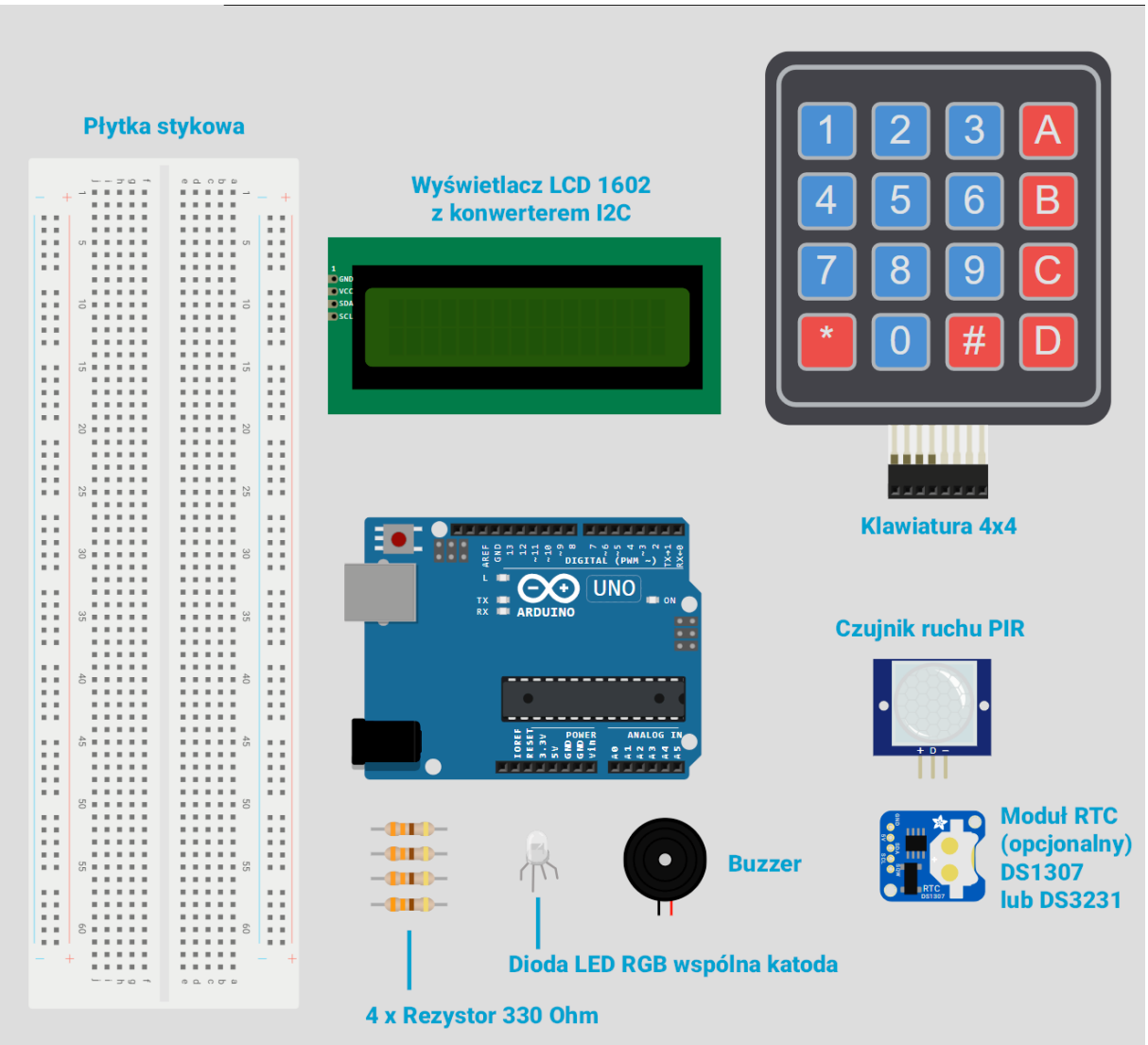
1. Stan alarmu: WYLACZONY.
2. Wciśnij * – na wyświetlaczu pokaże się legenda znaczenia klawiszy.
3. Wciśnij #, aby wrócić do ekranu głównego.

Instrukcja składania

Uwagi początkowe

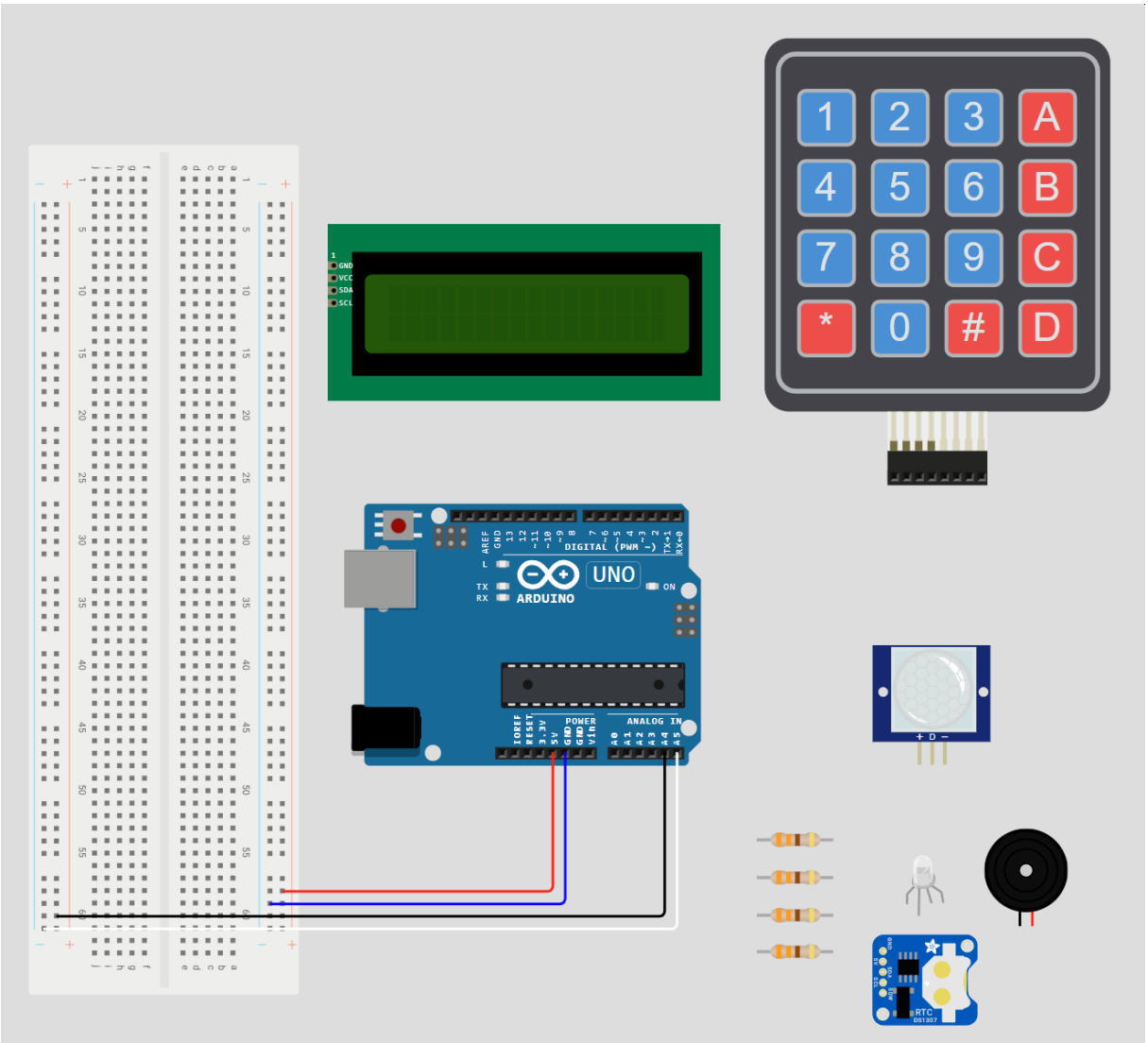
Moduł RTC jest opcjonalny. Płytka Arduino może być zamiennikiem. Diody LED i buzzer podłączamy przez rezystory. Kolejność pinów w klawiaturze może być inna niż przedstawiona w dokumentacji.

Komponenty



[1] Podłączenie płytki stykowej

Zaczynamy od podłączenia przewodów do płytki stykowej. Chcemy stworzyć 4 magistrale: plus, minus, SDA i SCL. Na poniższym obrazku widzimy, że do SDA wykorzystujemy wejście A4, a do SCL wejście A5. Arduino UNO R4 WiFi różni się tym, że ma porty dedykowane do SDA i SCL, ale jeśli nie masz ich na swojej płytce – skorzystaj z tych A4 i A5.



[2] Podłączenie diody LED RGB

W naszym przypadku jest to dioda ze wspólną katodą, oznacza to, że dłuższa nóżka jest minusem. Dioda ze wspólną katodą ma jedną niezaprzeczalną zaletę: nie musimy odejmować wartości kolorów od 255, co musielibyśmy robić w przypadku diody ze wspólną anodą.

Przykład:

Chcemy, aby dioda świeciła na pomarańczowy, który ma wartości RGB: 255 160 0. W przypadku naszej diody ze wspólną katodą – nic się nie zmienia. W programie przekazujemy dokładnie takie same wartości.

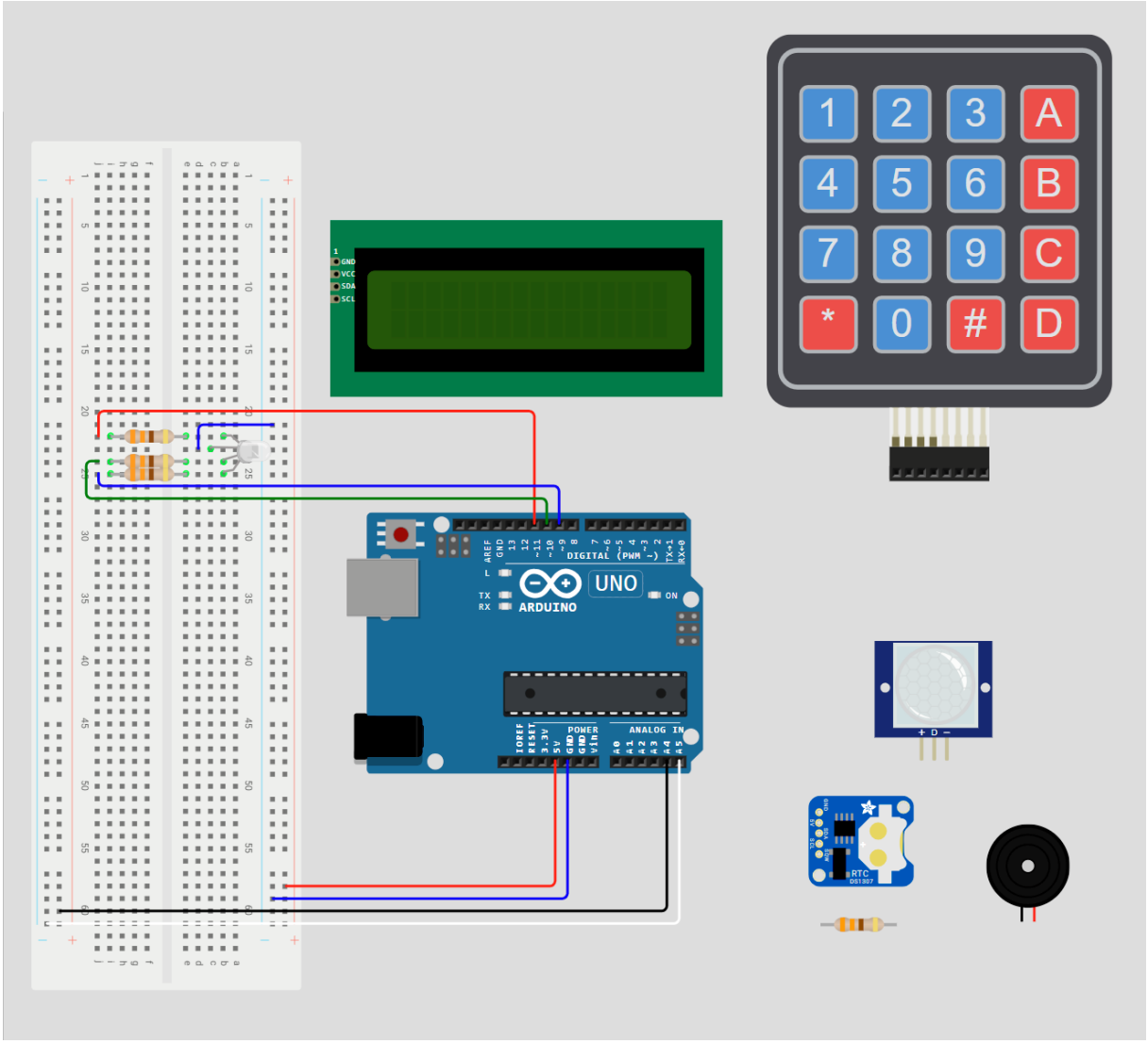
Jednak, jeśli mamy diodę ze wspólną anodą – musimy nasze wartości odjąć od 255. Czyli będzie to odpowiednio: 0 95 255, dlatego że stan wysoki jest zerem, a stan niski jedynką.

Pamiętaj o podłączeniu krótszych nóżek diody przez rezystory. To bardzo ważne. W przeciwnym wypadku popłynie przez nie duży prąd, który może je spalić.

Piny diody LED ze wspólną katodą.

Krótsza nóżka (1)	Dłuższa nóżka (2)	Krótsza nóżka (3)	Krótsza nóżka (4)
Red	Minus	Green	Blue

Nóżka R jest poprowadzona przez rezystor do wejścia D11 (PWM).
Nóżka G jest poprowadzona przez rezystor do wejścia D10 (PWM).
Nóżka B jest poprowadzona przez rezystor do wejścia D9 (PWM).



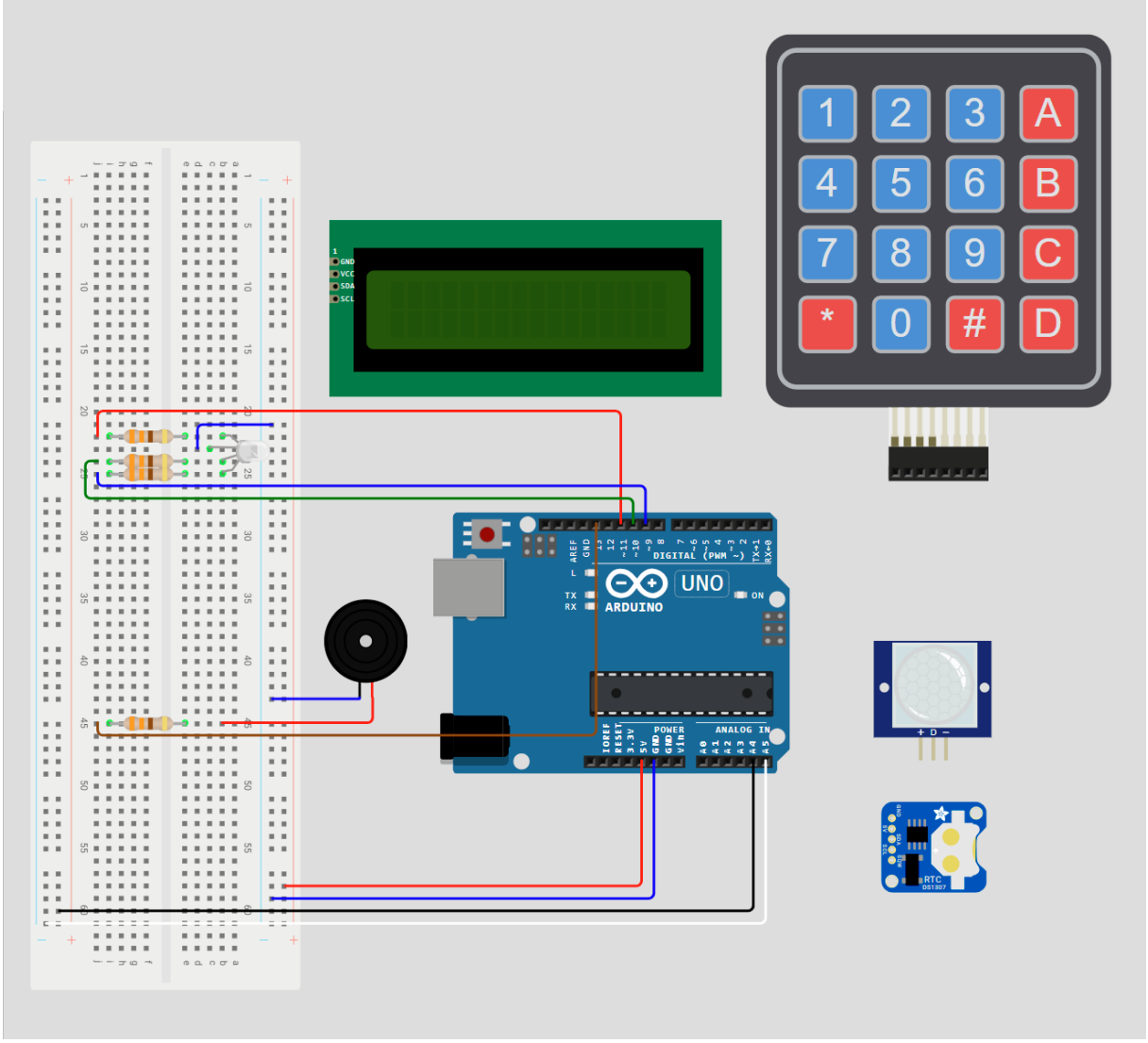
[3] Podłączenie buzzera

Buzzer, podobnie jak dioda LED musi być podłączony przez rezystor.

Dłuższa nóżka to plus, krótsza to minus. Jeśli nóżki są tej samej długości – zobacz nadrukowane oznaczenie na plastikowej części buzzera.

Plus buzzera podłączamy do wejścia D13 naszego Arduino.

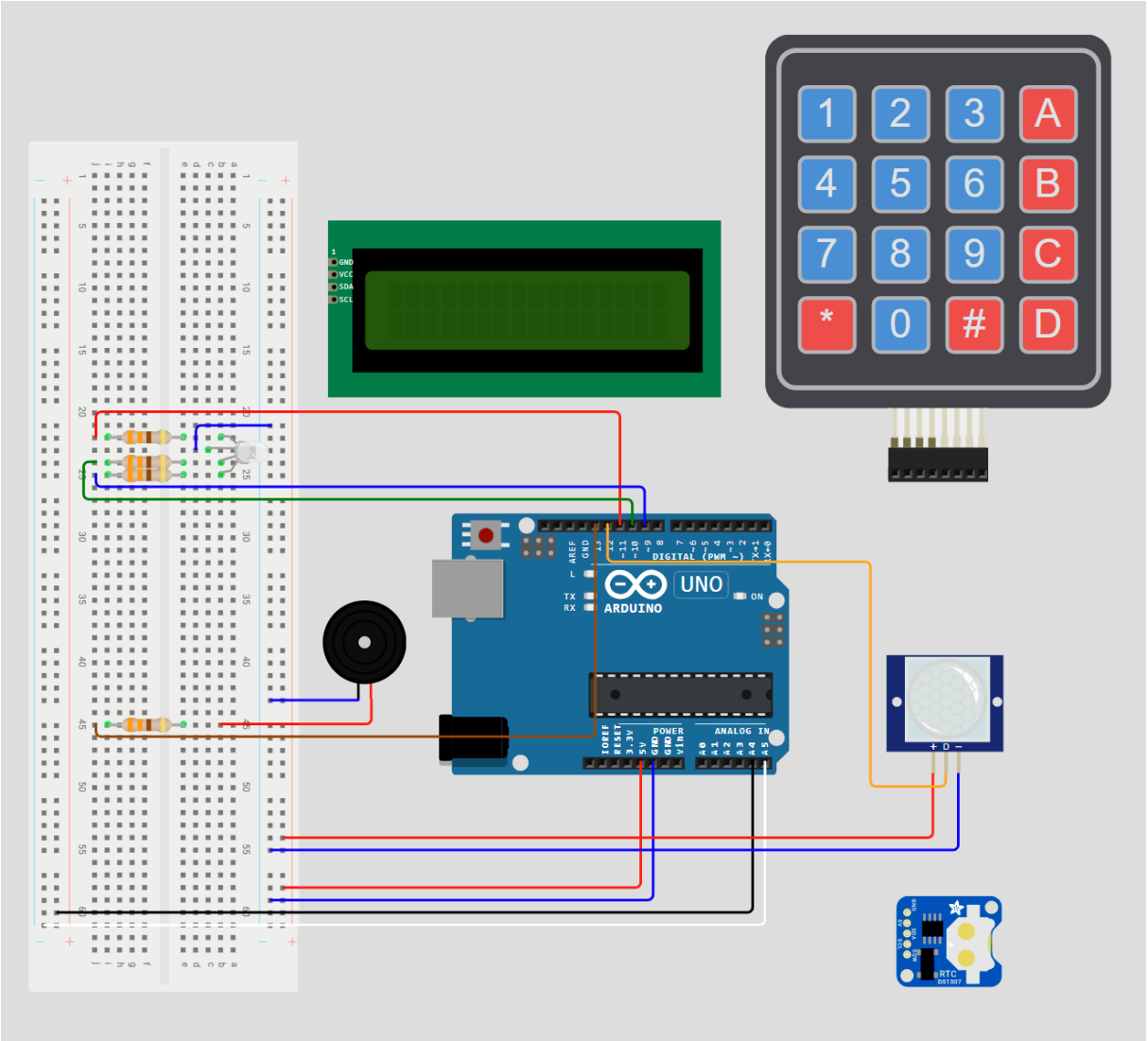
+	-
Przez rezystor do Arduino wejście D13	Minus na płytce stykowej



[4] Podłączenie czujnika ruchu PIR

Plus do szyny z plusami, Minus do szyny z minusami, a środkowy pin D podłączamy do wejścia D12 naszego Arduino.

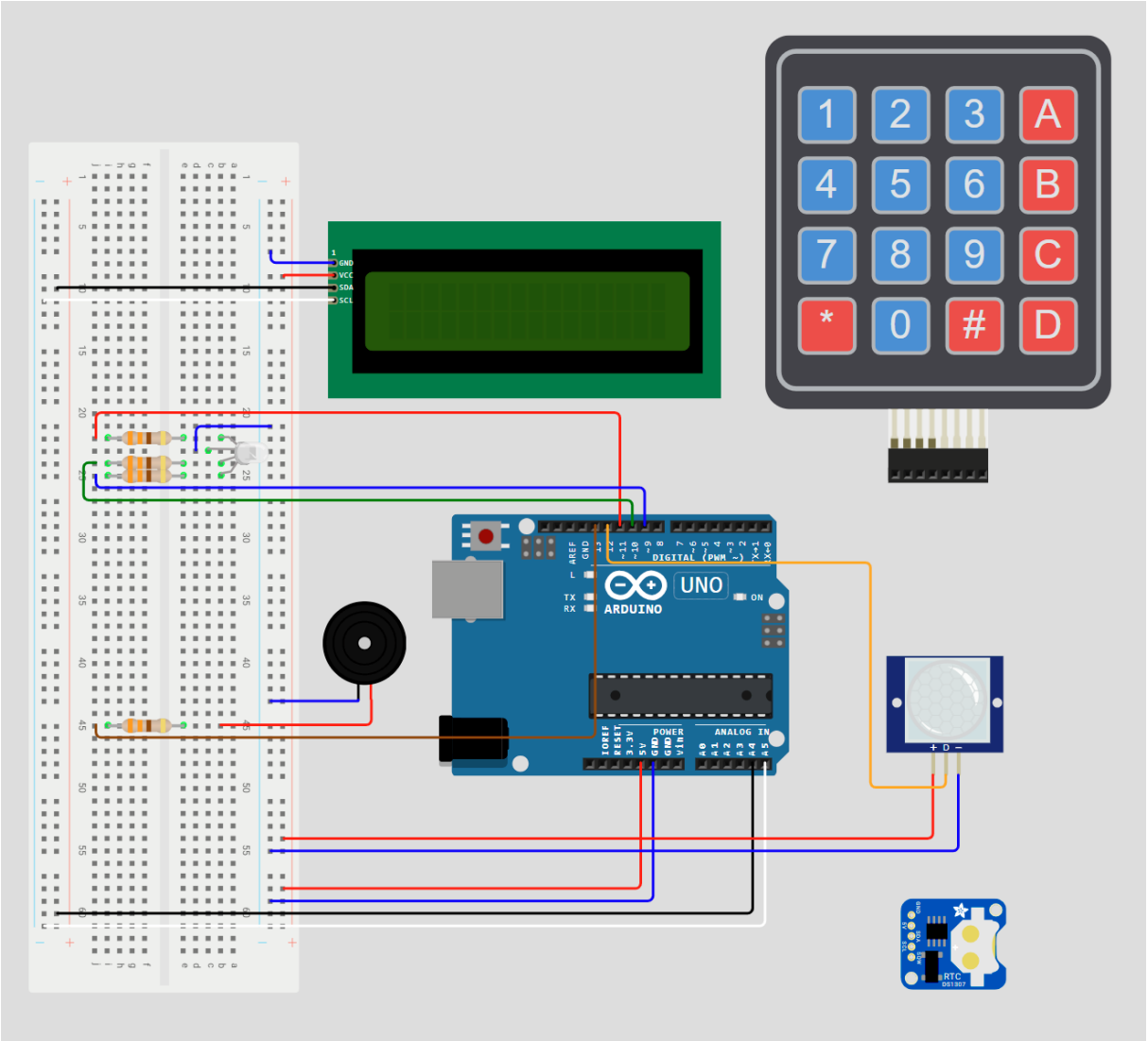
+	D	-
Plus	Wyjście D12 Arduino	Minus



[5] Podłączenie wyświetlacza LCD

Podłączenie wyświetlacza jest bardzo łatwe. Podłączamy go tylko i wyłącznie do płytki stykowej.

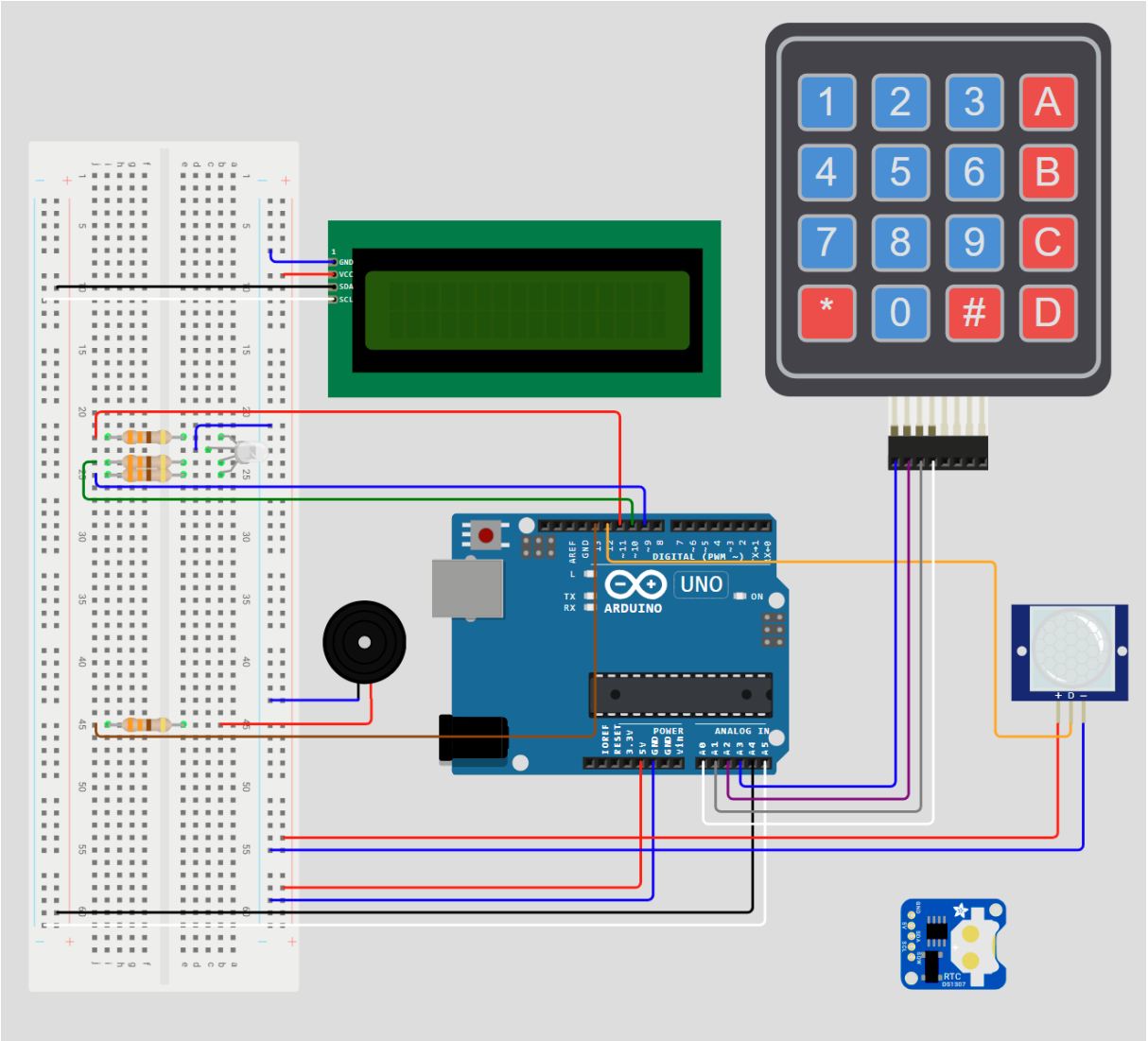
GND	VCC	SDA	SCL
Minus na płytce stykowej	Plus na płytce stykowej	Magistrala SDA na płytce stykowej	Magistrala SCL na płytce stykowej



[6] Podłączenie klawiatury

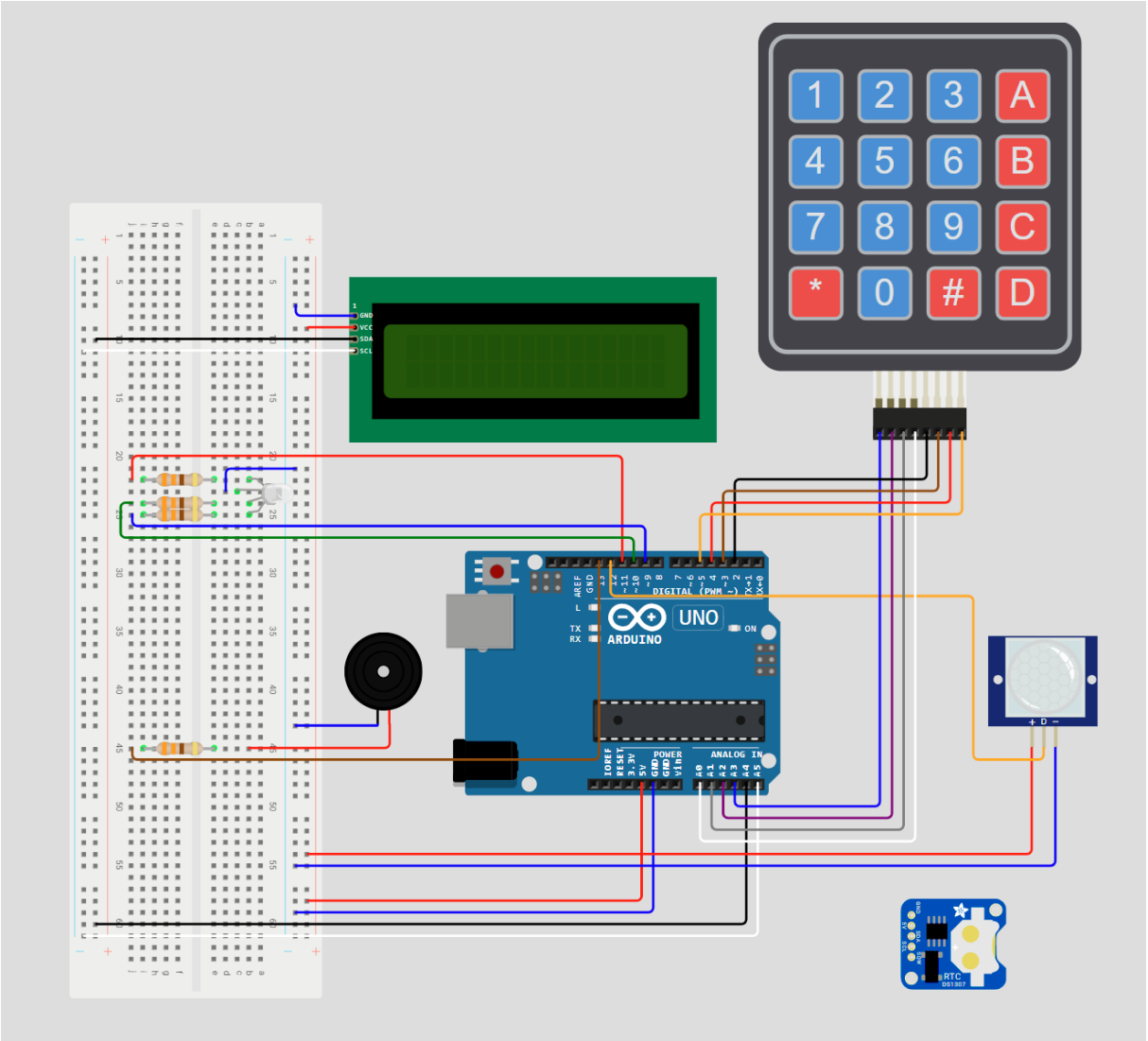
Podłączenie klawiatury potrafi być wyzwaniem. Producenci stosują różną kolejność pinów. W moim przypadku zaczyna się od 4 pinów R, a następnie idą piny C. Jeśli podłączysz tak, jak ja i wciskane klawisze nie będą pasowały – możesz spróbuj innych kombinacji, np. pierwsze po lewej jako piny C.

R1	**R2**	**R3**	**R4**
A3	A2	A1	A0



A teraz czas na kolejne cztery piny. W moim przypadku będą to piny C.

C1	**C2**	**C3**	**C4**
D2	D3	D4	D5



[7] Podłączenie RTC

Tak, jak wspomniałem wcześniej – ten moduł jest opcjonalny i występuje w kilku wersjach.

W serwisie Wokwi jest on w wersji DS1307. Fizycznie mam go w wersji DS3231.

Zakomentuj lub odkomentuj części kodu odpowiedzialne za definicję tego modułu:

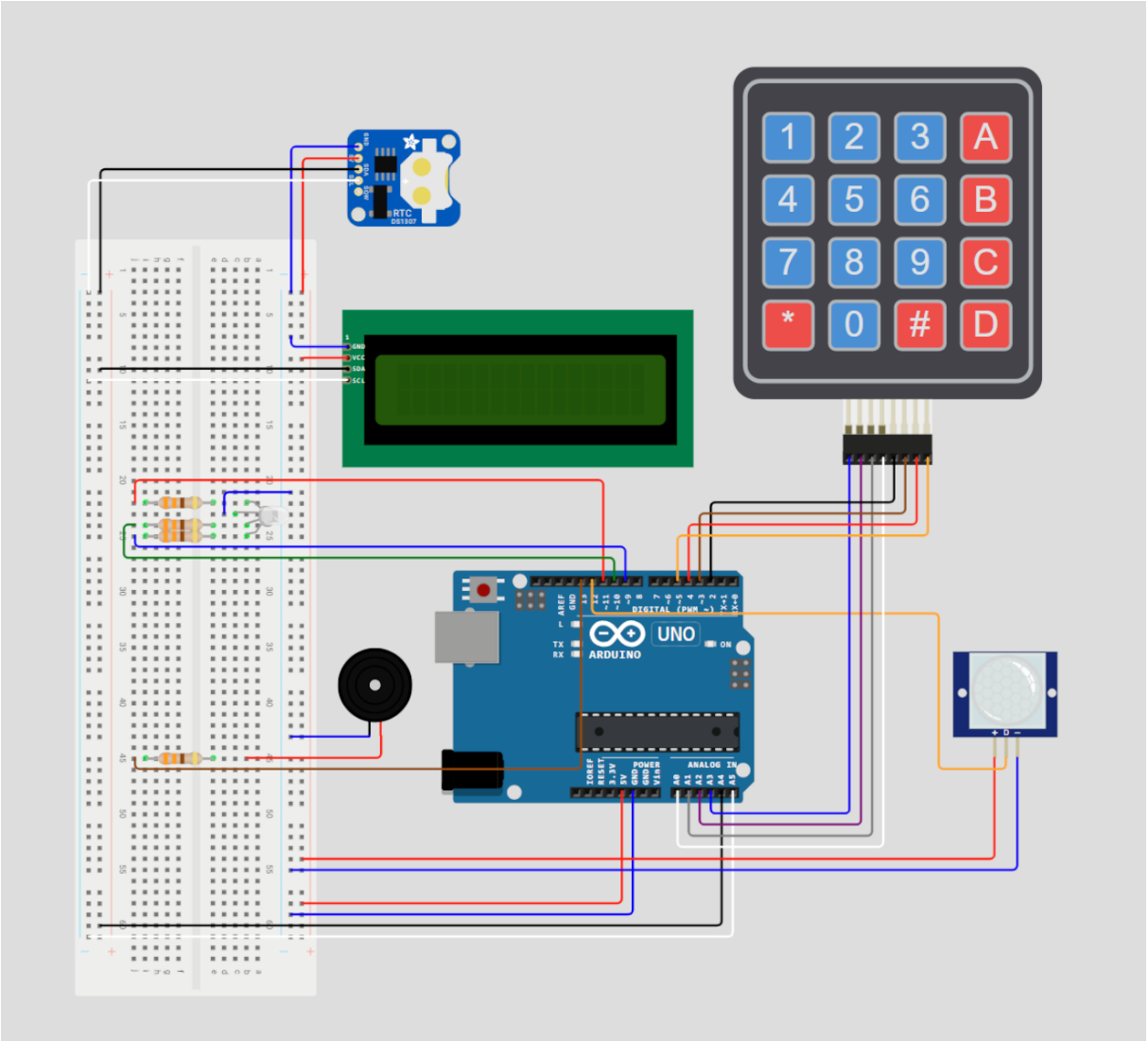
#define RTC_ON true lub false – ustaw true, aby włączyć korzystanie z tego modułu

#define RESET_TIME true lub false – ustaw true tylko za pierwszym razem, żeby zresetować zegar. Wgraj program, uruchom urządzenie, a następnie ustaw wartość na false i wgraj ponownie.

RTC_DS1307 rtc; lub RTC_DS3231 rtc; - zakomentuj urządzenie, którego nie masz.

Podłączenie jest proste, dokładnie jak wyświetlacz LCD.

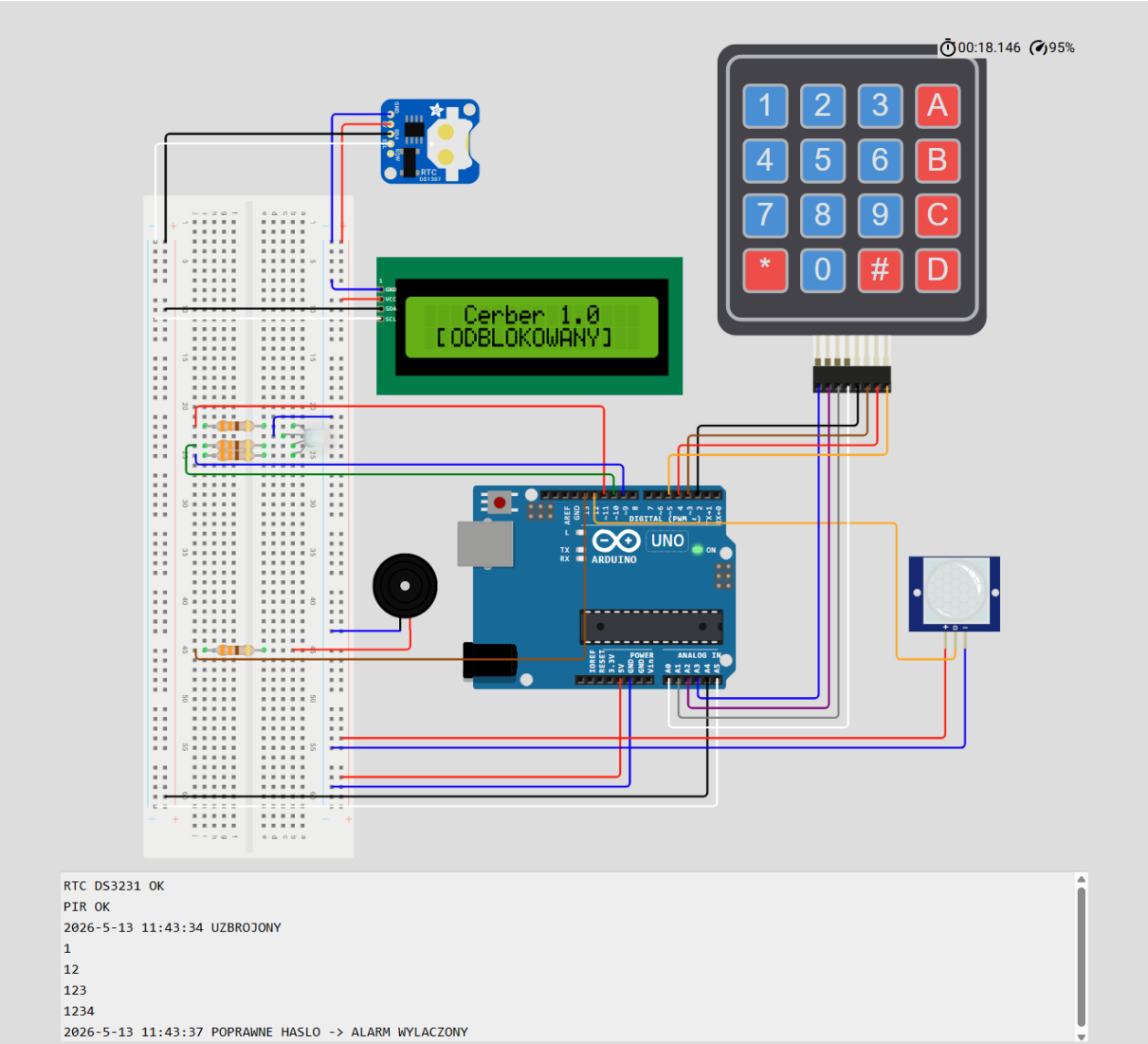
GND	5V	SDA	SCL
Minus	Plus	Magistrala SDA na płytce stykowej	Magistrala SCL na płytce stykowej



Działający system

Jeśli dotrwałeś do tej pory – oznacza to, że możesz mieć działający system.

Jeśli chcesz zobaczyć jak ja to wykonałem – zapraszam Ciebie do mojego projektu na Wokwi: https://wokwi.com/projects/463992320389011457



Możliwości dalszego rozwoju

- wykorzystanie WiFi
- wymiana informacji przez MQTT (lekki protokół komunikacyjny oparty na modelu publikacja/subskrypcja, zaprojektowany specjalnie dla urządzeń IoT i M2M. Działa na zasadzie brokera, który pośredniczy w wymianie komunikatów między klientami, co zapewnia niskie zużycie pasma i wysoką niezawodność w niestabilnych sieciach)
- powiadomienia SMS
- połączenie z aplikacją mobilna
- RFID
- NFC
- logi na karcie pamięci
- kamera
- zdalne sterowanie
- czujniki okien / drzwi